1. HTTP State, Sessions, and Authentication

Research Topic: How HTTP applications preserve state across request-response cycles.

HTTP is fundamentally a "stateless" protocol, meaning every request from a client to a server is treated as a completely new interaction with no memory of previous ones. To create a seamless user experience—such as staying logged into the Sticky Notes app—developers use session management. When a user authenticates (logs in), the server generates a unique **Session ID** and stores it in its database or memory. This ID is sent back to the user's browser, usually stored as a **Cookie**.

For every subsequent request, the browser automatically attaches this cookie. The server receives the ID, looks it up in its session store, and "remembers" who the user is. In Django, this is handled by the `SessionMiddleware` and `AuthenticationMiddleware` configured in `settings.py`. These components act as a bridge, intercepting requests to verify credentials before the view logic even runs. Without this mechanism, users would have to re-enter their username and password for every single page click or note edit, as the server would have no way to link the "Update Note" request to the "Login" request that happened moments before.

2. Migrating Django to MariaDB

Research Topic: Procedures for performing Django database migrations to a server-based relational database like MariaDB.

Transitioning from a local SQLite development environment to a robust, server-based relational database like **MariaDB** requires a specific procedural workflow to ensure data integrity. First, the developer must install a database driver, such as mysqlclient, which allows Python to communicate with MariaDB. Next, the DATABASES dictionary in settings.py must be updated to change the ENGINE to django.db.backends.mysql and provide the server's credentials, including the database name, user, password, host, and port.

Once the connection is configured, the migration process begins with python manage.py makemigrations. This command analyzes the current Django models and packages any changes into migration files. The final step is running python manage.py migrate, which instructs Django to read those files and generate the equivalent SQL statements for MariaDB, physically building the tables, columns, and relationships (like ForeignKeys) on the remote server. This ensures the database schema precisely matches the application's object-oriented structure. Using a server-based system like MariaDB is essential for production environments because it handles concurrent user connections and large datasets much more efficiently than a local file-based SQLite database